

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_095035a0-e9f\agent-a6d28c27b4b4f879d.jsonl`  
- SHA-256: `cb1c7a7a37495d21dcd065f25cebb80c7a6a58cd453b4755e4fe15c57b108749`  
- Source modified: `2026-06-13T19:15:53+00:00`  
- Imported at: `2026-07-08T16:00:00+00:00`  
- project: `wf_095035a0-e9f`  
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-13T19:13:53.215Z)

Adversarially VERIFY this fusion-P2 review finding by reading the actual code. Try to REFUTE it; only confirm real=true if it genuinely holds against the code. Default to real=false if uncertain or if it's a nitpick/false-positive.

```
FINDING: {"title":"WASB shuttle X/Y and person bbox are in DIFFERENT coordinate spaces ?  
shuttle_player_colocation likely ?0 always (spatial  
misalignment)","severity":"high","file":"backend/pipeline/detectors/fusion_features.py:154-185  
(consumed via backend/pipeline/segmenters/fusion_hybrid.py:98-106; trajectory produced by  
backend/pipeline/detectors/wasb_infer.py:223,337-341)","detail":"detections_per_frame returns  
frame_width/frame_height = the ORIGINAL video resolution (cap.get(CAP_PROP_FRAME_WIDTH/HEIGHT),  
person_detector.py:282-283) and the person bboxes are in original-video pixels. But the WASB  
trajectory's .x/.y come from the WASB tracker (wasb_infer.py:339 `r.get('x')`/`r.get('y')`),  
which operates in the model's OUTPUT heatmap resolution (`output_wh = (out_width, out_height)`,  
wasb_infer.py:223) ? NOT the original video pixels. _shuttle_in_box (fusion_features.py:154-161)  
compares the shuttle point against the person box DIRECTLY in pixels (`(x1-px) <= sx <=  
(x2+px)`), so a shuttle coordinate in ~512x288 space tested against a person box in ~1920x1080  
(or 3840x2160 4K) space will essentially never be inside/adjacent. The TEMPORAL alignment that  
detections_per_frame is responsible for (frame-index ? .frame) IS correct, but the spatial  
colocation feature ? the headline 'held-shuttle discriminator' (fusion_features.py:166-170) ?  
would silently return ~0.0 on real footage, and the pure unit tests in  
tests/test_fusion_features.py can't catch it because they hand-craft shuttle coords in the same  
space as the boxes. NOTE: confirming the exact WASB output scale requires the WASB-SBDT repo in  
WSL (not in this repo), so this is GPU/WSL-side to fully verify ? but the code as written  
assumes shuttle and person share a pixel space, and nothing rescales the shuttle by  
(orig_w/out_w, orig_h/out_h).","fix":"Establish ONE shared coordinate space before colocation:  
either (a) normalise the shuttle x/y to 0-1 using the WASB output_wh and normalise the person  
foot/box to 0-1 using the original frame dims, then compare in normalised space; or (b) rescale  
the WASB trajectory x/y into original-video pixels (multiply by orig_w/out_w, orig_h/out_h) at  
parse time. Add a fusion_features test that places the shuttle in a DIFFERENT numeric range than  
the boxes to prove the spaces are reconciled. Have the owner confirm the WASB tracker's output  
resolution on a real clip."}
```

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:13:56.230Z)

I'll verify this finding by reading the actual code. Let me start by pulling the latest and examining the relevant files.

3. user (2026-06-13T19:13:57.215Z)

```
1      """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3      NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4      math is unit-testable in a GPU-free dev container. The detection (person boxes,
5      shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6      module only turns those signals into a handful of scale/translation-invariant
7      features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8      memorise this court/camera and fail to generalise off a small labelled set).
9
10     Conventions:
11     - A person box is `(x1, y1, x2, y2)` in pixels; `(track_id, box)` when motion
needs it.
12     - `per_frame` is `{frame_idx: [(track_id, box), ...]}` keyed by the ORIGINAL video
frame
13     index (so it aligns directly with the shuttle trajectory's `frame` ? no clip re-
timing).
14     - A shuttle point has `frame`, `visible`, `x`, `y` (pixels) ? the WASB
trajectory shape.
15     - `court_polygon` is normalised 0-1 `(x, y), ...)` or `None` (None ? no mask,
every
16     detection counts ? the player-count-only mode).
17     - `net` is `(axis 'x'|'y', position 0-1 normalised)` or `None` (None ? two-sided =
0.0).
18
19     Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20     floats in [0, 1] except `mean_player_motion` (a non-negative normalised speed).
21     """
22     from __future__ import annotations
23
24     from typing import Dict, List, Optional, Sequence, Tuple
25
26     BBox = Tuple[float, float, float, float]
27     Net = Tuple[str, float]
28
29
30     def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31         """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32         (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33         if polygon is None or len(polygon) < 3:
34             return False
35         x, y = point
36         inside = False
37         n = len(polygon)
38         j = n - 1
39         for i in range(n):
40             xi, yi = polygon[i]
41             xj, yj = polygon[j]
42             # Does the horizontal ray at y cross edge (i, j)?
43             if (yi > y) != (yj > y):
44                 x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45                 if x < x_cross:
46                     inside = not inside
47             j = i
48         return inside
49
50
```

```

51     def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52         """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53         the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54         if frame_width <= 0 or frame_height <= 0:
55             return (0.0, 0.0)
56         x1, _y1, x2, y2 = box
57         return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60     def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61         if frame_width <= 0 or frame_height <= 0:
62             return (0.0, 0.0)
63         x1, y1, x2, y2 = box
64         return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67     def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                        court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69         """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70         True (count everyone) when it is None."""
71         if court_polygon is None:
72             return True
73         return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
74
75
76     def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
77                        frame_height: float,
78                        court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
79         """Per-frame count of in-court persons (one entry per sampled frame)."""
80         return [
81             sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
court_polygon))
82             for _frame_idx, dets in sorted(per_frame.items())
83         ]
84
85
86     def _median(vals: Sequence[float]) -> float:
87         if not vals:
88             return 0.0
89         s = sorted(vals)
90         n = len(s)
91         return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
92
93
94     def players_in_court(counts: Sequence[int]) -> float:
95         """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
96         return _median(list(counts))
97
98
99     def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
100         """Fraction of sampled frames with >= `min_players` in court (track stability vs
flicker)."""
101         if not counts:
102             return 0.0
103         return sum(1 for c in counts if c >= min_players) / len(counts)
104
105
106     def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
107                           frame_width: float, frame_height: float) -> float:
108         """Mean per-track centre displacement between consecutive sampled frames, normalised
109         by the frame width (resolution-invariant). 0.0 if <2 frames or no shared tracks."""
110         if frame_width <= 0 or len(per_frame) < 2:

```

```

111         return 0.0
112     frames = sorted(per_frame.keys())
113     steps: List[float] = []
114     for a, b in zip(frames, frames[1:]):
115         prev = {tid: box for tid, box in per_frame[a]}
116         for tid, box in per_frame[b]:
117             if tid in prev:
118                 cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
119                 cx1, cy1 = _center_norm(box, frame_width, frame_height)
120                 steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
121     if not steps:
122         return 0.0
123     return sum(steps) / len(steps)
124
125
126 def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
127                     frame_height: float, net: Optional[Net],
128                     court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
129     """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
130
131     A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
132     The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
133     0.0.
134
135     """
136     if net is None or not per_frame:
137         return 0.0
138     axis, pos = net
139     both = 0
140     for _frame_idx, dets in per_frame.items():
141         near, far = False, False
142         for _tid, box in dets:
143             if not person_in_court(box, frame_width, frame_height, court_polygon):
144                 continue
145             fx, fy = _foot_norm(box, frame_width, frame_height)
146             coord = fx if axis == "x" else fy
147             if coord < pos:
148                 near = True
149             else:
150                 far = True
151         if near and far:
152             both += 1
153     return both / len(per_frame)
154
155 def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
156                    pad_frac: float) -> bool:
157     """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
158     width/height (the 'adjacent / in-hand' tolerance)?"""
159     x1, y1, x2, y2 = box
160     px = pad_frac * frame_width
161     py = pad_frac * frame_height
162     return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
163
164 def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
165                             frame_width: float, frame_height: float,
166                             pad_frac: float = 0.03) -> float:
167     """Fraction of (visible shuttle frames that were also person-sampled) in which the
168     shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
169     ~0**
170     ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
171     """
172     if not per_frame or frame_width <= 0 or frame_height <= 0:

```

```

172         return 0.0
173     aligned = 0
174     colocated = 0
175     for p in shuttle_points or []:
176         if not getattr(p, "visible", False):
177             continue
178         dets = per_frame.get(int(p.frame))
179         if dets is None:          # shuttle frame wasn't person-sampled ? can't judge
180             continue
181         aligned += 1
182         if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
183             for _tid, box in dets):
184             colocated += 1
185     return colocated / aligned if aligned else 0.0
186
187
188     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
189     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
190     PERSON_FEATURE_NAMES = [
191         "players_in_court", "two_sided_motion", "mean_player_motion",
192         "in_court_ratio", "shuttle_player_colocation",
193     ]
194
195
196     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
197 shuttle_points,
198                                frame_width: float, frame_height: float,
199                                court_polygon: Optional[Sequence[Tuple[float, float]]] =
200 None,
201                                net: Optional[Net] = None,
202                                min_players: int = 2) -> Dict[str, float]:
203         """Aggregate the per-window person features from per-frame detections + the in-
204 window
205 shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
206         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
207         return {
208             "players_in_court": players_in_court(counts),
209             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
210 court_polygon),
211             "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
212             "in_court_ratio": in_court_ratio(counts, min_players),
213             "shuttle_player_colocation": shuttle_player_colocation(
214                 per_frame, shuttle_points, frame_width, frame_height),
215         }

```

4. user (2026-06-13T19:13:57.520Z)

```

1     """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3     WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4     own building blocks (ImageDataset transform + detector + online tracker) to run on
5     any folder of extracted frames and emit a trajectory CSV in the exact shape the
6     indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8         Frame,Visibility,X,Y
9
10    It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11    imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12    (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14    Resumable across reboots
15    -----
16    The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17    the slow, expensive part; the CPU tracker pass that turns detections into a

```

```

18 trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19 resume mid-stream). We exploit that split:
20
21 * The detector pass writes its per-window raw outputs incrementally to
22 ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23 records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24 most ``--flush-every`` windows of GPU work instead of the whole pass.
25 * On restart we replay the cached windows, skip the DataLoader ahead to the first
26 un-cached window, and continue. Once every window is cached, the detector pass
27 is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28 the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29 not the GPU hour.
30
31 The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32 to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33 weights, sport, window size, or frame count change.
34
35 Usage (in WSL, from ~/models/WASB-SBDT/src):
36 python wasb_infer.py --frames_dir /path/frames --weights
37 ../pretrained_weights/wasb_badminton_best.pth.tar \
38 --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
39 every 1000] [--fresh]
40
41 """
42 import os
43 import os.path as osp
44 import csv
45 import glob
46 import json
47 import argparse
48 import logging
49 from collections import defaultdict
50
51 logger = logging.getLogger(__name__)
52
53 CACHE_VERSION = 1
54 _DET_FILE = "det_raw.jsonl"
55 _MANIFEST_FILE = "manifest.json"
56
57 def _build_cfg(weights: str, sport: str):
58     """Compose the WASB eval config via Hydra, overriding model/weights/device."""
59     from hydra import compose, initialize
60     with initialize(config_path="configs", version_base=None):
61         cfg = compose(config_name="eval", overrides=[
62             f"dataset={sport}",
63             "model=wasb",
64             f"detector.model_path={weights}",
65             "runner.device=cuda",
66             "runner.gpus=[0]", # this box has a single GPU; default config assumes
67             more
68         ])
69     return cfg
70
71 def _frame_id(path: str) -> int:
72     """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
73     180)."""
74     stem = osp.splitext(osp.basename(path))[0]
75     digits = "".join(ch for ch in stem if ch.isdigit())
76     return int(digits) if digits else -1
77
78 # ----- #
79 # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
80 # ----- #

```

```

79     def _cache_paths(cache_dir: str):
80         return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
81
82
83     def default_cache_dir(out_csv: str) -> str:
84         """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
85         d = osp.dirname(osp.abspath(out_csv))
86         stem = osp.splitext(osp.basename(out_csv))[0]
87         return osp.join(d, stem + "_wasbcache")
88
89
90     def _atomic_write_text(path: str, text: str) -> None:
91         """Write then rename so a crash mid-write can't leave a half-written file."""
92         tmp = path + ".tmp"
93         with open(tmp, "w") as f:
94             f.write(text)
95             f.flush()
96             os.fsync(f.fileno())
97         os.replace(tmp, path)
98
99
100    def _det_plain(det) -> list:
101        """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
102        list."""
103        xy = det["xy"]
104        return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
105
106    def _dets_to_tracker(plain_list):
107        """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
108        array,
109        the tracker does vector math / np.linalg.norm on it)."""
110        import numpy as np
111        return [{"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
112                for p in plain_list]
113
114    def write_manifest(cache_dir: str, manifest: dict) -> None:
115        _, mpath = _cache_paths(cache_dir)
116        _atomic_write_text(mpath, json.dumps(manifest, indent=2))
117
118
119    def read_manifest(cache_dir: str):
120        _, mpath = _cache_paths(cache_dir)
121        if not osp.exists(mpath):
122            return None
123        try:
124            with open(mpath) as f:
125                return json.load(f)
126        except (json.JSONDecodeError, OSError):
127            return None
128
129
130    def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
131        frames_in: int, frames_total: int) -> bool:
132        """A cache is reusable only if the run that produced it matches this run's
133        inputs."""
134        if not manifest or manifest.get("version") != CACHE_VERSION:
135            return False
136        return (
137            manifest.get("frames_dir") == osp.abspath(frames_dir)
138            and manifest.get("weights") == weights
139            and manifest.get("sport") == sport
140            and manifest.get("frames_in") == frames_in
141            and manifest.get("frames_total") == frames_total

```

```

141     )
142
143
144 def reset_cache(cache_dir: str) -> None:
145     """Drop any existing cache so the next run starts clean."""
146     det_jsonl, mpath = _cache_paths(cache_dir)
147     for p in (det_jsonl, mpath):
148         if osp.exists(p):
149             os.remove(p)
150
151
152 def load_and_clean_cache(cache_dir: str):
153     """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
154     rebuild the per-frame detection accumulation, and rewrite the file to exactly that
155     prefix so a trailing half-written line from a crash can't corrupt future appends.
156
157     Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
158     [x, y, score, scale] candidates (accumulated across every window the frame is in,
159     in window order ? identical to a non-resumed run).
160     """
161     det_jsonl, _ = _cache_paths(cache_dir)
162     det_by_fid = defaultdict(list)
163     valid: list = []
164     if osp.exists(det_jsonl):
165         with open(det_jsonl, "r") as f:
166             for line in f:
167                 s = line.strip()
168                 if not s:
169                     continue
170                 try:
171                     obj = json.loads(s)
172                 except json.JSONDecodeError:
173                     break # truncated trailing line (crash mid-
flush)
174                 if obj.get("w") != len(valid): # non-contiguous -> stop at the gap
175                     break
176                 valid.append(s)
177                 for fid, plain in obj["f"]:
178                     det_by_fid[int(fid)].extend([list(p) for p in plain])
179                 # Guarantee a clean append boundary by rewriting only the good prefix.
180                 _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
181     return len(valid), det_by_fid
182
183
184 def _append_windows(fh, objs) -> None:
185     """Append window records as JSONL and durably flush (bounded loss on crash)."""
186     for o in objs:
187         fh.write(json.dumps(o, separators=(",", ":")) + "\n")
188     fh.flush()
189     os.fsync(fh.fileno())
190
191
192 def _atomic_write_trajectory(out_csv: str, rows) -> None:
193     os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
194     tmp = out_csv + ".tmp"
195     with open(tmp, "w", newline="") as f:
196         w = csv.writer(f)
197         w.writerow(["Frame", "Visibility", "X", "Y"])
198         w.writerows(rows)
199         f.flush()
200         os.fsync(f.fileno())
201     os.replace(tmp, out_csv)
202
203
204 # ----- #

```



```

205 # Inference
206 # ----- #
207 def run(frames_dir: str, weights: str, out_csv: str, sport: str = "badminton",
208         limit: int = 0, batch_size: int = 8, num_workers: int = 4,
209         log_every_batches: int = 50, cache_dir: "str | None" = None,
210         flush_every: int = 1000, fresh: bool = False) -> str:
211     import time
212     import torch
213     from torch.utils.data import DataLoader
214     from detectors import build_detector
215     from trackers import build_tracker
216     from dataloaders import build_img_transforms, build_seq_transforms
217     from dataloaders.dataset_loader import ImageDataset
218     from utils import Center
219
220     cfg = _build_cfg(weights, sport)
221     frames_in = int(cfg["model"]["frames_in"])
222     input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
223     output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
224
225     frames = sorted(glob.glob(osp.join(frames_dir, "*.png"))) +
glob.glob(osp.join(frames_dir, "*.jpg")))
226     if limit:
227         frames = frames[:limit]
228     if len(frames) < frames_in:
229         raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in
{frames_dir}")
230
231     # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
checkpoint).
232     samples = []
233     for i in range(len(frames) - frames_in + 1):
234         window = frames[i:i + frames_in]
235         annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
for p in window]
236         samples.append({"frames": window, "annos": annos})
237     windows_total = len(samples)
238
239     # ---- cache setup / resume decision ----- #
240     if cache_dir is None:
241         cache_dir = default_cache_dir(out_csv)
242     os.makedirs(cache_dir, exist_ok=True)
243     det_jsonl, _ = _cache_paths(cache_dir)
244     manifest = None if fresh else read_manifest(cache_dir)
245     resume = manifest_compatible(
246         manifest or {}, frames_dir=frames_dir, weights=weights, sport=sport,
247         frames_in=frames_in, frames_total=len(frames),
248     )
249     if resume:
250         windows_done, det_by_fid = load_and_clean_cache(cache_dir)
251         logger.info(f"resuming from cache: {windows_done}/{windows_total} windows
already detected")
252     else:
253         if manifest is not None:
254             logger.info("cache incompatible with this run -> starting fresh")
255             reset_cache(cache_dir)
256             windows_done, det_by_fid = 0, defaultdict(list)
257
258     base_manifest = {
259         "version": CACHE_VERSION,
260         "frames_dir": osp.abspath(frames_dir),
261         "weights": weights,
262         "sport": sport,
263         "frames_in": frames_in,
264         "frames_total": len(frames),

```

```

265     "windows_total": windows_total,
266     "windows_done": windows_done,
267 }
268 write_manifest(cache_dir, base_manifest)
269
270 # ---- detector pass over the un-cached windows ----- #
271 remaining = samples[windows_done:]
272 if remaining:
273     _, transform_test = build_img_transforms(cfg)
274     try:
275         _, seq_transform_test = build_seq_transforms(cfg)
276     except Exception:
277         seq_transform_test = None
278     ds = ImageDataset(cfg, remaining, input_wh, output_wh,
279                      transform=transform_test, seq_transform=seq_transform_test,
is_train=False)
280     loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
num_workers=num_workers)
281     detector = build_detector(cfg)
282
283     t0 = time.time()
284     done = windows_done
285     win_idx = windows_done
286     log_every = max(1, log_every_batches)
287     flush_n = max(1, flush_every)
288     pending = []
289     logger.info(f"detecting on {len(remaining)} remaining windows "
290               f"(total {windows_total}, batch={batch_size},
workers={num_workers})...")
291     det_f = open(det_jsonl, "a")
292     try:
293         with torch.no_grad():
294             for bi, batch in enumerate(loader):
295                 imgs, _hms, trans = batch[0], batch[1], batch[2]
296                 img_paths = [list(t) for t in batch[-1]] # [frame_in][batch] ->
path
297
298                 batch_results, _ = detector.run_tensor(imgs, trans)
299                 nb = imgs.shape[0]
300                 for ib in range(nb):
301                     frames_payload = []
302                     for ie in sorted(batch_results[ib].keys()):
303                         p = img_paths[ie][ib]
304                         fid = _frame_id(p)
305                         plain = [_det_plain(d) for d in batch_results[ib][ie]]
306                         frames_payload.append([fid, plain])
307                         det_by_fid[fid].extend(plain)
308                     pending.append({"w": win_idx, "f": frames_payload})
309                     win_idx += 1
310                 done += nb
311                 if len(pending) >= flush_n or done >= windows_total:
312                     _append_windows(det_f, pending)
313                     pending = []
314                     base_manifest["windows_done"] = done
315                     write_manifest(cache_dir, base_manifest)
316                 if (bi + 1) % log_every == 0 or done >= windows_total:
317                     el = time.time() - t0
318                     new_done = done - windows_done
319                     rate = new_done / el if el > 0 else 0.0
320                     eta = (windows_total - done) / rate if rate > 0 else 0.0
321                     logger.info(f"{done}/{windows_total} "
322                               f"({100 * done // max(1, windows_total)}%) |
{rate:.1f} win/s | "
323                               f"elapsed {el:.0f}s | ETA {eta:.0f}s")
324
325             if pending:
326                 _append_windows(det_f, pending)

```

```

325         base_manifest["windows_done"] = done
326         write_manifest(cache_dir, base_manifest)
327     finally:
328         det_f.close()
329         logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
330 else:
331     logger.info("detector cache already complete -> tracker-only re-run")
332
333 # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
334 tracker = build_tracker(cfg)
335 tracker.refresh()
336 rows = []
337 for fid in sorted(det_by_fid.keys()):
338     r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
339     rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
340
341 _atomic_write_trajectory(out_csv, rows)
342 visible = sum(1 for _, v, *_ in rows if v)
343 logger.info(f"wrote {len(rows)} rows -> {out_csv}")
344 logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
345 return out_csv
346
347
348 def extract_frames(video_path: str, frames_dir: str) -> str:
349     """Decode a video to PNG frames named by source frame index (00000000.png ...).
350
351     Idempotent: if the dir already holds the expected number of frames (matching the
352     container's reported frame count) we skip re-decoding ? re-extraction after a
353     reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
354     extracting with ffmpeg on the host and passing --frames_dir.
355     """
356     import cv2
357     os.makedirs(frames_dir, exist_ok=True)
358     cap = cv2.VideoCapture(video_path)
359     if not cap.isOpened():
360         raise SystemExit(f"cannot open video: {video_path}")
361     expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
362     existing = len(glob.glob(osp.join(frames_dir, "*.png")))
363     if expected > 0 and existing >= expected:
364         cap.release()
365         logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
366         return frames_dir
367     i = 0
368     while True:
369         ok, frame = cap.read()
370         if not ok:
371             break
372         cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
373         i += 1
374     cap.release()
375     logger.info(f"extracted {i} frames -> {frames_dir}")
376     return frames_dir
377
378
379 def main():
380     ap = argparse.ArgumentParser()
381     ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
382     ap.add_argument("--video", help="video file; frames are extracted internally (in
WSL)")
383     ap.add_argument("--frames_out_dir", help="where to extract frames when --video is
used")
384     ap.add_argument("--weights", required=True)
385     ap.add_argument("--out", required=True)
386     ap.add_argument("--sport", default="badminton")
387     ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick

```

```

tests)")
388     ap.add_argument("--batch-size", type=int, default=8)
389     ap.add_argument("--num-workers", type=int, default=4)
390     ap.add_argument("--log-every-batches", type=int, default=50, help="progress log
cadence")
391     ap.add_argument("--cache-dir", default=None,
392                     help="resumable detector cache dir (default: <out>_wasbcache next to
--out)")
393     ap.add_argument("--flush-every", type=int, default=1000,
394                     help="fsync the detector cache every N windows (bounds reboot
loss)")
395     ap.add_argument("--fresh", action="store_true", help="ignore any existing cache and
start over")
396     args = ap.parse_args()
397
398     frames_dir = args.frames_dir
399     if args.video:
400         frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
401     if not frames_dir:
402         raise SystemExit("provide --frames_dir or --video")
403     run(frames_dir, args.weights, args.out, args.sport, args.limit,
404         batch_size=args.batch_size, num_workers=args.num_workers,
405         log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
406         flush_every=args.flush_every, fresh=args.fresh)
407
408
409     if __name__ == "__main__":
410         main()
411

```

5. assistant (2026-06-13T19:14:00.296Z)

Now let me look at how `detections\_per\_frame` produces `frame\_width`/`frame\_height` and how `fusion\_hybrid.py` consumes everything, plus the person_detector.

6. user (2026-06-13T19:14:01.749Z)

```

1     """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3     Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4     TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5     do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6     shuttle-seeded window via two overridable seams the engine exposes:
7
8     - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9     GPU-free, computed from the trajectory we already have); and, only when the person cue
10    is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11    (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12    fusion (P3). The person path lazily builds ONE `PersonDetector` ? so the default path
13    never imports torch / touches the GPU.
14    - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
15    (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
16    dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
17    **Persisted candidates remain the full superset** regardless, so the offline
experiment
18    harness can re-score any variant.
19
20    Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
21    nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
22    and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the

```

```

default
23     segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
24     experiment harness (EXPERIMENT_HARNESS.md), never silently.
25     """
26     from typing import Any, Dict, List, Optional, Tuple
27
28     from backend.pipeline.segmenters.base import segmenter_registry
29     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
30     from backend.pipeline.detectors.base import DetectorRunner
31     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
32     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
33     from backend.pipeline.detectors import rally_gate
34     from backend.pipeline.detectors import fusion_features as ff
35
36
37     class FusionSegmenter(TrajectoryHybridSegmenter):
38         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
39
40         family = "fusion"
41         display_label = "Fusion"
42
43         def __init__(self, db: Any, config: Any):
44             super().__init__(db, config)
45             # Built lazily only if the person cue is enabled (keeps torch/GPU off the
default path).
46             self._person: Optional[Any] = None
47
48             @property
49             def name(self) -> str:
50                 return "fusion_hybrid"
51
52             @property
53             def description(self) -> str:
54                 return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
55                         "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
56                         "AI provider sets semantic boundaries.")
57
58             def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
59                 substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
60                 if substrate == "tracknet":
61                     cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
62                     return TrackNetRunner(cfg), {
63                         "weights_path": cfg.weights_path,
64                         "queue_length": cfg.queue_length,
65                         "fusion_substrate": "tracknet",
66                     }
67                 wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
68                 return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
69
70             def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
71                                         frame_width: float, video_path: str,
72                                         idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
73                 stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
74                 # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
75                 # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
76                 # over the whole trajectory by rally_gate (camera-agnostic).

```

```

77         gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
min_crossings=0)
78         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
79
80         fcfg = idx_cfg.get("fusion", {})
81         if fcfg.get("cue_flags", {}).get("person", False):
82             stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
83         return stats
84
85     def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
86                         frame_width: float, video_path: str,
87                         fcfg: Dict[str, Any]) -> Dict[str, float]:
88         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
89         frame range and compute the scale/translation-invariant person features. Lazily
builds
90         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
91         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
92         from backend.pipeline.detectors.person_detector import PersonDetector
93         if self._person is None:
94             self._person = PersonDetector(self.config)
95
96         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
97         start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)
98         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
99         fw = det["frame_width"] or frame_width
100        fh = det["frame_height"] or 0.0
101
102        poly = fcfg.get("court_polygon")
103        polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
104        net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
105        shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
106        return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
107                                         court_polygon=polygon, net=net)
108
109    def _select_for_handoff(self, windows: List[Dict[str, Any]],
110                          window_stats: List[Dict[str, Any]],
111                          idx_cfg: Dict[str, Any]) -> List[int]:
112        """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
113        (0) keep every window ? identical to the substrate. Persisted candidates are
114        untouched (full superset)."""
115        fcfg = idx_cfg.get("fusion", {})
116        min_crossings = int(fcfg.get("min_crossings", 0))
117        min_players = int(fcfg.get("min_players", 0))
118        person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120        keep: List[int] = []
121        for i, st in enumerate(window_stats):
122            if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                continue
124            if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125                continue
126            keep.append(i)
127        return keep
128
129
130    segmenter_registry.register(FusionSegmenter)
131

```

```

1      """Person-detection rally cue ? torchvision detector + ByteTrack + motion windowing.
2
3      Extracted verbatim from `yolo_hybrid` (fusion P1, `docs/MULTI_SIGNAL_FUSION_PLAN.md`):
4      the **person cue is a swappable producer**, not logic baked into one segmenter. Today
5      `HybridYoloSegmenter` is the only consumer (it delegates Stage-1 here, byte-
6      identically);
7      next the fusion segmenter consumes the SAME producer over shuttle-seeded windows, and
8      later the learned classifier reads its persisted features. Keeping detection here (not
9      in
10     the segmenter) is what makes "add a cue = register a producer" true.
11
12     Behaviour is unchanged from the in-lined version ? only the home moved:
13     - `lazy_load_model`           ? load a permissively-licensed torchvision detector
14     - `make_tracker`             ? a fresh supervision ByteTrack per chunk
15     - `detect_and_track`         ? one frame ? confident persons with persistent IDs
16     - `extract_action_windows_from_chunk` ? a chunk ? high-motion candidate windows
17       with the motion stats (`peak/mean_velocity`, `active_frame_count`, `track_id_count`)
18       that feed the candidate `stats` JSON and the learned fusion features.
19
20     Heavy deps (torchvision, supervision) are imported lazily inside the methods so
21     importing
22     the segmenter registry never pulls torch on a machine that lacks it (and tests can mock
23     the seams). Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT
24     (ADR-005; ultralytics' AGPL YOLO was dropped).
25     """
26     import logging
27     from typing import Any, Dict, List, Optional, Tuple
28
29     import cv2
30     import torch
31
32     from backend.utils.geometry import get_velocity_normalised
33
34     logger = logging.getLogger(__name__)
35
36     # torchvision detection models are trained on COCO where the "person" category is
37     # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
38     # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
39     _PERSON_CLASS_ID = 1
40
41     # Permissively-licensed torchvision detectors selectable via config
42     # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
43     _DETECTOR_FACTORIES = {
44         "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
45         "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
46         "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
47     }
48     _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"
49
50     class PersonDetector:
51         """Stage-1 person detection + tracking + motion-window extraction (a rally cue).
52
53         Owns the torchvision model + per-chunk ByteTrack tracker. Constructed with the run
54         ``config`` (reads ``indexing.{use_gpu, detector_model, detector_score_threshold,
55         min_action_window_duration}``); the model loads lazily on first use.
56         """
57
58         def __init__(self, config: Dict[str, Any]):
59             self.config = config
60             # Dynamic torchvision module (None until lazy_load_model); typed Any so the
61             # `self.model([tensor])` inference call type-checks without a quarantine.
62             self.model: Any = None
63             self.device: Optional[str] = None

```

```

63     def lazy_load_model(self) -> None:
64         if self.model is not None:
65             return
66
67         # Imported lazily so the module imports without torchvision present and so
68         # the test suite (which pre-sets self.model) never pulls in model weights.
69         from torchvision.models import detection as tv_detection
70
71         idx_cfg = self.config.get("indexing", {})
72         use_gpu = idx_cfg.get("use_gpu", True)
73         self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
74         if use_gpu and not torch.cuda.is_available():
75             logger.warning("use_gpu is True but CUDA is not available. Falling back to
CPU.")
76
77         model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
78         if model_name not in _DETECTOR_FACTORIES:
79             logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
{_DEFAULT_DETECTOR}.")
80             model_name = _DEFAULT_DETECTOR
81
82         builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
83         logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
84         # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
85         self.model = builder(weights="DEFAULT")
86         self.model.eval()
87         self.model.to(self.device)
88
89     def make_tracker(self, fps: float) -> Any:
90         """Create a fresh ByteTrack tracker for one chunk (track IDs are chunk-local).
91
92         Imported lazily so tests can mock this method without supervision installed.
93         """
94         import supervision as sv
95         frame_rate = max(1, int(round(fps)))
96         return sv.ByteTrack(frame_rate=frame_rate)
97
98     def detect_and_track(self, frame: Any, tracker: Any,
99                         score_thresh: float) -> List[Tuple[int, Tuple[float, float,
float, float]]]:
100         """Detect persons in a frame and assign persistent track IDs.
101
102         Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
103         This replaces the old ultralytics ``model.track(...)`` call, which bundled
104         detection AND association: torchvision does the detection, supervision's
105         ByteTrack does the association.
106         """
107         import supervision as sv
108
109         # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
110         rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
111         tensor = torch.from_numpy(rgb).permute(2, 0,
1).float().div(255.0).to(self.device)
112         with torch.no_grad():
113             outputs = self.model([tensor])
114
115         out = outputs[0]
116         boxes = out["boxes"].detach().cpu().numpy()
117         labels = out["labels"].detach().cpu().numpy()
118         scores = out["scores"].detach().cpu().numpy()
119
120         # Keep only confident person detections.
121         mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
122         if not mask.any():
123             return []

```



```

124
125         # ByteTrack requires confidence to be populated (it filters on it internally).
126         dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
class_id=labels[mask])
127         tracked = tracker.update_with_detections(dets)
128
129         result: List[Tuple[int, Tuple[float, float, float, float]]] = []
130         for i in range(len(tracked)):
131             tid = tracked.tracker_id[i]
132             if tid is None:
133                 continue
134             x1, y1, x2, y2 = tracked.xyxy[i]
135             result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))
136         return result
137
138     def extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
139                                         velocity_thresh: float, merge_gap: float,
140                                         frame_skip: int = 3,
141                                         chunk_index: Optional[int] = None) ->
List[Dict[str, Any]]:
142         """
143         Runs person detection + tracking on a video chunk and extracts high-motion
144         action windows. Returns a list of dicts in the full video's timeframe, each
with:
145             start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
146             track_id_count, chunk_index.
147
148         Args:
149             frame_skip: Process every Nth frame (1 = every frame). Higher values
150                         dramatically reduce inference cost with minimal quality loss.
151             chunk_index: Index of the source chunk (recorded on each window for
traceability).
152         """
153         cap = cv2.VideoCapture(chunk_path)
154         if not cap.isOpened():
155             logger.error(f"Could not open {chunk_path}")
156             return []
157
158         fps = cap.get(cv2.CAP_PROP_FPS)
159         if fps <= 0:
160             fps = 30.0
161
162         frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
163         if frame_width <= 0:
164             frame_width = 1280.0 # Sensible fallback
165
166         # Effective sampling rate after frame-skipping
167         effective_fps = fps / max(frame_skip, 1)
168
169         # A fresh tracker per chunk ? track IDs only need to be consistent within a
170         # chunk (windows are computed per chunk, then merged across chunks by time).
171         tracker = self.make_tracker(effective_fps)
172         score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
173
174         # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
175         # so windows can be enriched with motion stats for logging + fine-tuning.
176         active_frames = []
177         frame_idx = 0
178         track_history: Dict[int, Tuple[float, float, float, float]] = {}
179
180         while True:
181             ret, frame = cap.read()
182             if not ret:
183                 break

```

```

184
185     # Skip frames for performance ? at 30fps, every 3rd frame still
186     # gives ~10 samples/second, plenty for human-scale velocity.
187     if frame_idx % max(frame_skip, 1) != 0:
188         frame_idx += 1
189         continue
190
191     # Detect persons + assign persistent track IDs.
192     detections = self.detect_and_track(frame, tracker, score_thresh)
193
194     frame_peak_velocity = 0.0
195     frame_active_ids = set()
196     current_ids = set()
197
198     for track_id, bbox in detections:
199         current_ids.add(track_id)
200
201         if track_id in track_history:
202             prev_bbox = track_history[track_id]
203             velocity = get_velocity_normalised(prev_bbox, bbox, effective_fps,
frame_width)
204             if velocity > velocity_thresh:
205                 frame_peak_velocity = max(frame_peak_velocity, velocity)
206                 frame_active_ids.add(track_id)
207
208             track_history[track_id] = bbox
209
210     # Prune stale IDs that are no longer visible ? prevents false
211     # velocity spikes when the tracker reassigns an old ID.
212     track_history = {k: v for k, v in track_history.items() if k in current_ids}
213
214     if frame_active_ids:
215         active_frames.append((frame_idx, frame_peak_velocity, frame_active_ids))
216
217     frame_idx += 1
218
219 cap.release()
220
221 # Group active frames into continuous windows
222 if not active_frames:
223     return []
224
225 max_gap_frames = int(merge_gap * fps)
226
227 def _finalize(group: List[Tuple[int, float, set]]) -> Dict[str, Any]:
228     velocities = [v for _, v, _ in group]
229     track_ids = set().union(*[ids for _, _, ids in group])
230     return {
231         "start": group[0][0] / fps + chunk_start,
232         "end": group[-1][0] / fps + chunk_start,
233         "active_frame_count": len(group),
234         "peak_velocity": max(velocities),
235         "mean_velocity": sum(velocities) / len(velocities),
236         "track_id_count": len(track_ids),
237         "chunk_index": chunk_index,
238     }
239
240 windows = []
241 group = [active_frames[0]]
242 for af in active_frames[1:]:
243     if af[0] - group[-1][0] <= max_gap_frames:
244         group.append(af)
245     else:
246         windows.append(_finalize(group))
247         group = [af]

```

```

248         windows.append(_finalize(group))
249
250         # Filter out extremely short action windows
251         min_window_duration = self.config.get("indexing",
252         {}).get("min_action_window_duration", 2.0)
253         windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
254         return windows
255
256     def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
257                             frame_skip: int = 3) -> Dict[str, Any]:
258         """Per-frame person detections over ``[start_frame, end_frame]`` of the ORIGINAL
259         video.
260
261         Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
262         ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and
263         discards the
264         per-frame boxes), this returns the raw per-frame detections so the fusion
265         features can
266         align them with the shuttle trajectory. It reads the source by **absolute frame
267         index**
268         (seek + sequential read), NOT a re-extracted clip, so the returned frame indices
269         match
270         the WASB trajectory's ``.frame`` exactly ? that alignment is what
271         ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
272
273         Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
274         "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-
275         seek
276         accuracy varies by codec (it lands near the requested keyframe then decodes
277         forward);
278         treat the indices as ±a few frames on real footage ? fine for window-level
279         features.
280
281         """
282         self.lazy_load_model()
283         cap = cv2.VideoCapture(video_path)
284         if not cap.isOpened():
285             logger.error(f"Could not open {video_path}")
286             return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
287
288         fps = cap.get(cv2.CAP_PROP_FPS)
289         if fps <= 0:
290             fps = 30.0
291         frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
292         frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
293         score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
294         0.5)
295
296         step = max(frame_skip, 1)
297         tracker = self.make_tracker(fps / step)
298
299         start_frame = max(0, int(start_frame))
300         frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
301         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
302         frame_idx = start_frame
303         while frame_idx <= end_frame:
304             ret, frame = cap.read()
305             if not ret:
306                 break
307             if (frame_idx - start_frame) % step == 0:
308                 frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
309                 frame_idx += 1
310         cap.release()
311         return {"frames": frames, "fps": fps,
312                 "frame_width": float(frame_width), "frame_height": float(frame_height)}
313

```

8. assistant (2026-06-13T19:14:08.782Z)

The finding has two parts: (1) temporal alignment is correct (the reviewer concedes this), and (2) spatial coordinate spaces differ between WASB shuttle X/Y and person bboxes. Let me verify the critical claim: does WASB emit coordinates in output-heatmap space (~512x288) rather than original-video pixels?

The key code is `wasb\_infer.py:337-341`: the tracker `r.get('x')`/`r.get('y')`. I need to understand the WASB tracker's coordinate scaling. The reviewer notes the WASB-SBDT repo is in WSL (not in this repo). Let me check what's available and also look at `parse\_trajectory\_csv` to understand how the shuttle X/Y flows and whether there's any rescaling. Let me also check how TrackNet (the alternative substrate) produces coords for comparison.

9. user (2026-06-13T19:14:09.889Z)

Found 18 files

```
tests\test_trajectory_hybrid_equivalence.py
tests\test_fusion_hybrid.py
backend\pipeline\segmenters\trajectory_hybrid.py
docs\CODE_MAP.md
backend\pipeline\detectors\wasb_runner.py
backend\pipeline\detectors\tracknet_runner.py
backend\pipeline\detectors\base.py
tests\test_tracknet_hybrid.py
backend\eval\calibrate_local.py
backend\eval\distill_local.py
backend\pipeline\detectors\wasb_infer.py
training\gen0\harness.py
tests\test_rally_seq_proto.py
backend\eval\rally_seq_proto.py
backend\eval\eval_partial_labels.py
backend\eval\calibrate_wasb.py
tests\test_tracknet_runner.py
backend\eval\export_wasb_segments.py
```

10. user (2026-06-13T19:14:10.243Z)

```
backend\pipeline\detectors\tracknet_runner.py:233: def parse_trajectory_csv(csv_win_path:
str) -> List[TrajectoryPoint]:
backend\pipeline\detectors\tracknet_runner.py:234:     """Parse a TrackNet predict CSV
backend\pipeline\detectors\tracknet_runner.py:235:     points: List[TrajectoryPoint] = []
backend\pipeline\detectors\tracknet_runner.py:236:     with open(csv_win_path, newline="") as
f:
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-237-         reader = csv.DictReader(f)
backend\pipeline\detectors\tracknet_runner.py-238-         for row in reader:
backend\pipeline\detectors\tracknet_runner.py-239-             try:
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-240-                 points.append(TrajectoryPoint(
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-241-                     frame=int(row["Frame"]),
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-242-                     visible=int(row["Visibility"]) == 1,
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-243-                     x=float(row["X"]),
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-244-                     y=float(row["Y"]),
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-245-                     ))
```

```

backend\pipeline\detectors\tracknet_runner.py-246-         except (KeyError, ValueError):
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-247-             continue
backend\pipeline\detectors\tracknet_runner.py-248-         points.sort(key=lambda p: p.frame)
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-249-         return points
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-250-
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-251-     @staticmethod
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-252-     def trajectory_to_action_windows(
backend\pipeline\detectors\tracknet_runner.py-253-         points: List[TrajectoryPoint],
backend\pipeline\detectors\tracknet_runner.py-254-         fps: float,
backend\pipeline\detectors\tracknet_runner.py-255-         frame_width: float,
backend\pipeline\detectors\tracknet_runner.py-256-         chunk_start: float = 0.0,
backend\pipeline\detectors\tracknet_runner.py-257-         velocity_thresh: float = 0.02,
backend\pipeline\detectors\tracknet_runner.py-258-         merge_gap: float = 2.0,
backend\pipeline\detectors\tracknet_runner.py-259-         min_window_duration: float = 2.0,
backend\pipeline\detectors\tracknet_runner.py-260-         chunk_index: Optional[int] = None,
backend\pipeline\detectors\tracknet_runner.py-261-     ) -> List[Dict[str, Any]]:
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-262-         """Convert a shuttle
trajectory into candidate rally windows.
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-263-
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-264-         A frame is "active" when the
shuttle is visible AND its inter-frame
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-265-         displacement (normalised by
frame width, per second) exceeds
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-266-         ``velocity_thresh`` ? i.e. the
shuttle is in flight. Active frames within
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-267-         ``merge_gap`` seconds of each
other are grouped into one window; short
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-268-         windows are dropped. Mirrors
HybridYoloSegmenter's windowing so the dict
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-269-         shape feeding the AI-handoff
phase is identical.
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-270-         """
backend\pipeline\detectors\tracknet_runner.py-271-         if fps <= 0:
C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\pipeline\detectors\tracknet_runner.py-272-             fps = 30.0
backend\pipeline\detectors\tracknet_runner.py-273-         if frame_width <= 0:

```

11. assistant (2026-06-13T19:14:16.430Z)

Now let me look at how `velocity\_thresh` is used in `trajectory\_to\_action\_windows` ? the shuttle velocity is normalized by `frame\_width`. This is a critical clue: if the shuttle X/Y were already in output-heatmap space (~512 wide) but normalized by the original `frame\_width` (~1920), the velocity normalization would also be wrong. But the existing WASB substrate works (it's the shipped default). Let me check how velocity is computed and what `frame\_width` is passed.

Let me also check the WASB tracker source. The reviewer says it's in WSL. But let me check the calibrate/export scripts and how the WASB X/Y is interpreted, and check if there's any reference to the actual output resolution or rescaling anywhere.

12. user (2026-06-13T19:14:17.884Z)

```

273         if frame_width <= 0:
274             frame_width = 1280.0
275
276         # Compute per-frame normalised velocity from the last *visible* point.
277         active = [] # (frame_idx, velocity)
278         prev: Optional[TrajectoryPoint] = None
279         for p in points:
280             if not p.visible:
281                 prev = None # break the trajectory; absent shuttle = not in flight
282                 continue
283             if prev is not None:
284                 dframes = p.frame - prev.frame
285                 if dframes > 0:
286                     dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
287                     velocity = (dist / frame_width) * (fps / dframes)
288                     if velocity > velocity_thresh:
289                         active.append((p.frame, velocity))
290             prev = p
291
292         if not active:
293             return []
294
295         max_gap_frames = int(merge_gap * fps)
296
297         def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
298             vels = [v for _, v in group]
299             return {
300                 "start": group[0][0] / fps + chunk_start,
301                 "end": group[-1][0] / fps + chunk_start,
302                 "active_frame_count": len(group),
303                 "peak_velocity": max(vels),
304                 "mean_velocity": sum(vels) / len(vels),
305                 "track_id_count": 1, # single shuttle; kept for window-shape parity
306                 "chunk_index": chunk_index,
307             }
308
309         windows = []
310         group = [active[0]]
311         for af in active[1:]:
312             if af[0] - group[-1][0] <= max_gap_frames:
313                 group.append(af)
314             else:
315                 windows.append(_finalize(group))
316                 group = [af]
317         windows.append(_finalize(group))
318
319         return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
320

```

13. user (2026-06-13T19:14:19.733Z)

```

tests\test_config.py:179:     assert validation.get("min_resolution_width") == 1280
tests\test_config.py:191:     assert cfg.get("min_resolution_width") == 1280 # lives inside
`validation`
tests\test_config_wasb.py:36:     # Must be DECLARED (nested sections drop unknown keys) and
default to 1280 ?
tests\test_config_wasb.py:40:     assert AppConfig().get("indexing")["ai_chunk_scale_width"] ==
1280
backend\config\models.py:25:     min_resolution_width: int = 1280
backend\config\models.py:136:     ai_chunk_scale_width: int = 1280
tests\test_fusion_hybrid.py:48:         patch.object(FusionSegmenter, "_probe_fps_width",
return_value=(30.0, 1280.0)), \
tests\test_gemini.py:108:     (default 1280) rather than stream-copied. Also fixes keyframe drift
at boundaries."""
tests\test_gemini.py:115:         assert call.kwargs["scale_width"] == 1280

```

```

backend\reporting\harvest.py:207:    # used its internal fallback (30fps / 1280px) ? surface
that as the footgun it is,
backend\reporting\harvest.py:212:        vm["fps"] = 30.0 # matches the runners' fallback
(_probe_fps_width -> 30.0 / 1280)
backend\eval\audio\eval_probe.py:104:    ap.add_argument("--frame-width", type=float,
default=1920.0)
backend\eval\eval_partial_labels.py:25:        --fps 59.94 --frame-width 1920
backend\eval\eval_partial_labels.py:196:    ap.add_argument("--frame-width", type=float,
default=1920.0)
backend\eval\calibrate_wasb.py:10:        --labels "golden_labels_badminton.csv" --fps 59.94
--frame-width 1920
backend\eval\calibrate_wasb.py:100:    ap.add_argument("--frame-width", type=float,
default=1920.0)
backend\eval\gemini_refine.py:19:        --fps 59.94 --frame-width 1920 --out
output\adarsh_gemini.csv [--limit 4]
backend\eval\gemini_refine.py:73:        clip_scale_width: Optional[int] = 1280) ->
Tuple[int, Interval, List[Interval]]:
backend\eval\gemini_refine.py:104:        min_dur: float = 2.0, limit: int = 0,
clip_scale_width: Optional[int] = 1280) -> dict:
backend\eval\gemini_refine.py:150:        out_csv: str =
"output/gemini_fullvideo.csv", clip_scale_width: Optional[int] = 1280,
backend\eval\gemini_refine.py:204:    ap.add_argument("--frame-width", type=float,
default=1920.0)
backend\eval\gemini_refine.py:213:    ap.add_argument("--clip-scale-width", type=int,
default=1280,
backend\eval\export_wasb_segments.py:16:        --labels "Annotation
Setup/golden_labels_badminton.csv" --fps 59.94 --frame-width 1920
backend\eval\export_wasb_segments.py:202:    ap.add_argument("--frame-width", type=float,
default=1920.0)
tests\test_reporting.py:82:        {"codec_type": "video", "width": 1920, "height":
1080, "r_frame_rate": "30/1"},
tests\test_reporting.py:91:        {"codec_type": "video", "width": 1280, "height": 720,
"r_frame_rate": "30/1"},
tests\test_reporting_harvest.py:37:    "streams": [{"width": 1920, "height": 1080,
"r_frame_rate": "60000/1001", "codec_name": "hevc"}],
backend\pipeline\segmenters\gemini.py:117:        ai_scale_width =
int(idx_cfg.get("ai_chunk_scale_width", 1280) or 0)
backend\pipeline\segmenters\gemini.py:217:        f"Set
indexing.ai_chunk_scale_width (default 1280) so chunks are re-encoded/"
tests\test_tracknet_runner.py:72:    # 30 fps, 1280px wide. Shuttle moves ~60px/frame while
visible (frames 0..29) =>
tests\test_tracknet_runner.py:74:    fps, width = 30.0, 1280.0
tests\test_tracknet_runner.py:94:        pts, fps=30.0, frame_width=1280.0,
velocity_thresh=0.05,
tests\test_tracknet_runner.py:107:        burst1 + gap + burst2, fps=30.0, frame_width=1280.0,
backend\pipeline\validators\resolution_fps.py:42:        min_w =
val_cfg.get("min_resolution_width", 1280)
backend\pipeline\detectors\person_detector.py:164:        frame_width = 1280.0 # Sensible
fallback
tests\test_tracknet_hybrid.py:27:    fn = patch.object(TrackNetHybridSegmenter,
"_probe_fps_width", return_value=(30.0, 1280.0))(fn)
tests\test_tracknet_hybrid.py:164:        assert fps == 30.0 and width == 1280.0
backend\pipeline\detectors\tracknet_runner.py:274:        frame_width = 1280.0
backend\pipeline\detectors\wasb_infer.py:222:    input_wh = (int(cfg["model"]["inp_width"]),
int(cfg["model"]["inp_height"]))
backend\pipeline\detectors\wasb_infer.py:223:    output_wh = (int(cfg["model"]["out_width"]),
int(cfg["model"]["out_height"]))
backend\pipeline\segmenters\trajectory_hybrid.py:64:        return (fps if fps > 0 else 30.0,
width if width > 0 else 1280.0)
tests\test_validators_base.py:16:        context["ffprobe_meta"] = {"fps": 59.94, "width": 1920,
"height": 1080}
tests\test_trajectory_hybrid_equivalence.py:49:        patch.object(cls, "_probe_fps_width",
return_value=(30.0, 1280.0)), \
tests\test_trajectory_hybrid_equivalence.py:121:    stats = seg._enrich_candidate_stats(cw, [],
30.0, 1280.0, "v.mp4", {})

```

14. assistant (2026-06-13T19:14:29.338Z)

This is a crucial finding. The existing WASB substrate's `trajectory\_to\_action\_windows` (shared by both TrackNet and WASB via `tracknet\_runner.parse\_trajectory\_csv`) normalizes shuttle velocity by `frame\_width` at line 287: `velocity = (dist / frame\_width) \* (fps / dframes)`.

The calibration scripts (`calibrate\_wasb.py:100`, `export\_wasb\_segments.py:202`) default `--frame-width 1920` ? the ORIGINAL video width. This strongly implies the WASB trajectory X/Y the pipeline consumes is interpreted in **original-video pixel space** (1920-wide), NOT output-heatmap space. If WASB emitted ~512-wide coords, the velocity normalization with 1920 would make velocities ~3.75x too small and the existing (shipped, calibrated) WASB substrate would be broken ? but it's the calibrated default.

Let me verify the WASB tracker's actual coordinate output. The reviewer says the WASB-SBDT repo is in WSL. Let me check if it's accessible, and check the calibration doc / memory for what coordinate space was established.

15. user (2026-06-13T19:14:31.684Z)

NOT in this filesystem (WSL-side)
wsl not reachable here

16. user (2026-06-13T19:14:33.364Z)

No matches found

17. assistant (2026-06-13T19:14:39.786Z)

The WASB-SBDT repo is WSL-side (not accessible). Let me check the calibration memory / docs to see if the coordinate space was empirically established, and look at the `calibrate\_wasb.py` more carefully ? if it ran calibration with `--frame-width 1920` and got sensible velocity thresholds, that's strong evidence the X/Y are in original pixel space.

Let me check the WASB infer transform handling. The key question: in `wasb\_infer.py`, the `trans` matrix (`batch[2]`) passed to `detector.run\_tensor(imgs, trans)` ? does WASB's detector apply an inverse-transform back to original coords? Let me look at how `ImageDataset` and the transform produce `trans`, and whether the detector un-projects. Let me also check memory notes on calibration.

18. user (2026-06-13T19:14:40.601Z)

```
1      """Per-video calibration driver: WASB trajectory + golden rallies -> tuned thresholds +
"+X%".
2
3      Wires the already-tested pieces together:
4      WASB trajectory CSV (Frame,Visibility,X,Y)
5      -> TrackNetRunner.trajectory_to_action_windows(cfg)    [predict_fn]
6      -> backend.eval.calibration (improvement_report / cross_validate)
7
8      Run (after a full-video WASB pass exists):
9      python -m backend.eval.calibrate_wasb --trajectory full_traj.csv \
10         --labels "golden_labels_badminton.csv" --fps 59.94 --frame-width 1920
11
12      Honesty note: the grid is *selected* on the full GT, so `improvement_report` on that
13      same GT is an optimistic upper bound. The trustworthy number is the
14      cross-validated held-out recall (and, once a 2nd labelled video exists,
15      `leave_one_video_out` for precision/F1 generalization).
16      """
17      from __future__ import annotations
18
19      import argparse
20      from typing import List, Tuple, Callable
21
22      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
```



```

23     from backend.eval.calibration import (
24         select_best, improvement_report, cross_validate, evaluate,
25     )
26
27     Interval = Tuple[float, float]
28     Config = Tuple[float, float, float] # (velocity_thresh, merge_gap,
min_window_duration)
29
30     BASELINE: Config = (0.02, 2.0, 2.0) # the WasbConfig / trajectory_to_action_windows
defaults
31
32
33     def load_golden_gts(csv_path: str) -> List[Interval]:
34         """Rally [start,end] seconds from a golden CSV ? delegates to the canonical loader.
35
36         Accepts BOTH golden conventions now (start_time/end_time AND the collector export's
37         start,end ? the historical fork is closed; see gt_loader.py / ADR-008). Keeps this
38         driver's legacy lenient semantics (skip bad rows), but skipped rows are now logged.
39         """
40         from backend.eval.gt_loader import load_gt_intervals
41         return load_gt_intervals(csv_path, strict=False)
42
43
44     def make_predict_fn(trajectory_csv: str, fps: float, frame_width: float) ->
Callable[[Config], List[Interval]]:
45         """predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""
46         points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
47
48         def predict_fn(cfg: Config) -> List[Interval]:
49             vt, mg, mwd = cfg
50             wins = TrackNetRunner.trajectory_to_action_windows(
51                 points, fps=fps, frame_width=frame_width,
52                 velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
53             )
54             return [(w["start"], w["end"]) for w in wins]
55
56         return predict_fn
57
58
59     def default_grid() -> List[Config]:
60         return [(vt, mg, mwd)
61                 for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
62                 for mg in (1.0, 2.0, 3.0)
63                 for mwd in (1.0, 2.0, 3.0)]
64
65
66     def run(trajectory_csv: str, labels_csv: str, fps: float, frame_width: float,
67            iou: float = 0.5) -> dict:
68         gts = load_golden_gts(labels_csv)
69         if not gts:
70             raise SystemExit(f"no usable golden rallies in {labels_csv}")
71         predict_fn = make_predict_fn(trajectory_csv, fps, frame_width)
72         grid = default_grid()
73
74         base = evaluate(predict_fn(BASELINE), gts, iou)
75         best_cfg, best_f1 = select_best(predict_fn, grid, gts, metric="f1",
iou_threshold=iou)
76         fit = improvement_report(predict_fn, BASELINE, best_cfg, gts, iou) # OPTIMISTIC
(fit on full GT)
77         cv = cross_validate(predict_fn, grid, gts, k=5) # HONEST held-
out recall
78
79         print("=== WASB rally-segmentation calibration ===")
80         print(f"golden rallies: {len(gts)} | IoU thresh: {iou}")
81         print(f"baseline cfg {BASELINE}: precision={base.precision:.3f}")

```

```

recall={base.recall:.3f} "
82         f"f1={base.f1:.3f} meanIoU={base.mean_iou:.3f}")
83     print(f"tuned cfg      {best_cfg}: f1={best_f1:.3f}")
84     print(f"  precision {fit['baseline']]['precision']:.3f} ->
{fit['tuned']]['precision']:.3f} "
85         f"({fit['precision_delta_pct']:+.1f}% , fit-on-full = optimistic)")
86     print(f"    f1          {fit['baseline']]['f1']:.3f} -> {fit['tuned']]['f1']:.3f} "
87         f"({fit['f1_delta_pct']:+.1f}%)")
88     print(f"HONEST (cross-val, {cv.get('folds','?')} folds): held-out recall "
89         f"{cv.get('mean_test',float('nan')):.3f} ± {cv.get('std_test',0):.3f}, "
90         f"generalization_gap {cv.get('generalization_gap',float('nan')):+.3f}")
91     print("(For precision/F1 generalization, add a 2nd labelled video ->
leave_one_video_out.)")
92     return {"baseline": base.as_dict(), "best_cfg": best_cfg, "fit": fit, "cv": cv}
93
94
95     def main():
96         ap = argparse.ArgumentParser()
97         ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV
(Frame,Visibility,X,Y)")
98         ap.add_argument("--labels", required=True, help="golden labels CSV
(start_time/end_time cols)")
99         ap.add_argument("--fps", type=float, default=59.94)
100        ap.add_argument("--frame-width", type=float, default=1920.0)
101        ap.add_argument("--iou", type=float, default=0.5)
102        args = ap.parse_args()
103        run(args.trajectory, args.labels, args.fps, args.frame_width, args.iou)
104
105
106    if __name__ == "__main__":
107        main()
108

```

19. assistant (2026-06-13T19:14:46.934Z)

This is informative but velocity calibration with `frame\_width=1920` would still produce sensible rally windows even if X/Y were in 512-space, because velocity is a *threshold on relative motion* ? the absolute scale just shifts the optimal `velocity\_thresh` (and the grid would pick a smaller threshold). So calibration alone doesn't conclusively prove the coordinate space. I need the actual WASB output space.

Let me check the memory notes about WASB/TrackNet coordinate handling, and look for any documented evidence of the WASB output resolution. Let me search the calibration memory note and the WASB decision note.

20. user (2026-06-13T19:14:50.863Z)

C:/Users/avidu/.claude/projects/c--Users-avidu-Projects-badminton-highlight-indexer/memory/collector-indexer-bridge.md:12:Key reframing established 2026-05-26: the indexer's `yolo\_hybrid` segmenter uses **a stock person detector (person class only) + a velocity heuristic** ? there is no trained badminton model. (As of 2026-05-30 that detector is **torchvision Faster R-CNN + supervision ByteTrack**, swapped off AGPL Ultralytics YOLOv8 ? PR #8/ADR-005; the velocity-heuristic logic is unchanged.) The collector's data is **temporal-only** (rally `[start,end]`, stroke counts, scores; NO per-frame shuttle `(x,y)`). So this data trains/tunes the **temporal segmentation** stage, NOT a shuttle object detector (that needs coordinate labels ? see [[tracknet-wasb-wsl2-decision]] and [[candidate-segments-finetuning]]).

C:/Users/avidu/.claude/projects/c--Users-avidu-Projects-badminton-highlight-indexer/memory/current-state.md:244: `allowed\_video\_root` must contain `upload\_dir`). Original handoff context below:

C:/Users/avidu/.claude/projects/c--Users-avidu-Projects-badminton-highlight-indexer/memory/current-state.md:525: an unscoped new workstream (coordinate labels or detector swap); (5) OWNED_MODEL_IMPLEMENTATION_PLAN + NEXT_STEPS are

C:/Users/avidu/.claude/projects/c--Users-avidu-Projects-badminton-highlight-indexer/memory/current-state.md:1432: inp 512x288 (WxH). `dataloaders/\_\_init\_\_.py build\_img\_transforms`: T.ToTensor + Normalize

```

C:/Users/avidu/.claude/projects/c--Users-avidu-Projects-badminton-highlight-
indexer/memory/current-state.md:1435: run_tensor and used to map heatmap coords back to
original frame). So wrapper: per sliding window
C:/Users/avidu/.claude/projects/c--Users-avidu-Projects-badminton-highlight-
indexer/memory/current-state.md:1436: of 3 consecutive frames ? affine-resize to 512x288 ?
ToTensor+Normalize ? stack ? detector.run_tensor
C:/Users/avidu/.claude/projects/c--Users-avidu-Projects-badminton-highlight-
indexer/memory/current-state.md:1442: DETECTOR (needs x,y heatmap supervision). **So the rally
labels CANNOT fine-tune WASB's detector weights.**
C:/Users/avidu/.claude/projects/c--Users-avidu-Projects-badminton-highlight-
indexer/memory/current-state.md:1454: held-out golden rally IoU, coordinate ? pick max-precision
config. (Can't run before a trajectory exists.)
C:/Users/avidu/.claude/projects/c--Users-avidu-Projects-badminton-highlight-
indexer/memory/current-state.md:1513:- **Full-video WASB inference RUNNING** (task brmhazv6x;
orig b4pqhqkte died on reboot): `wasb_infer.py --frames_dir
C:/Users/avidu/.claude/projects/c--Users-avidu-Projects-badminton-highlight-
indexer/memory/tracknet-wasb-wsl2-decision.md:19:- **TrackNetV4** (TensorFlow,
https://github.com/AR4152/TrackNetV4) ? badminton-specific, highest reported shuttle accuracy,
ships `src/predict.py` that outputs per-frame (X,Y) coordinate CSV. Primary choice.
C:/Users/avidu/.claude/projects/c--Users-avidu-Projects-badminton-highlight-
indexer/memory/tracknet-wasb-wsl2-decision.md:28:**How to apply:** integrate via a **subprocess
adapter** ? a detector module beside [[candidate-segments-finetuning]] / yolo_hybrid that shells
out to the WSL `predict.py`, reads the coordinate CSV, and converts trajectory ? candidate
rallies feeding the existing stitching flow. Keeps TF dependency isolated. User is manually
annotating videos to fine-tune after integration.
===DOCS===
C:/Users/avidu/Projects/badminton-highlight-indexer/docs/archives/decisions/DECISIONS.md:23: -
**TrackNetV4** (TensorFlow) ? badminton-specific, heatmap + motion-attention,
C:/Users/avidu/Projects/badminton-highlight-indexer/docs/CODE_MAP.md:230:- `model=wasb` ->
`name=hrnet`, `frames_in=3`, `frames_out=3`, input 512x288 (WxH), ImageNet normalize; affine
resize maps heatmap coords -> original-frame coords. (model dims read from cfg["model"] at
runtime in wasb_infer.run.)
C:/Users/avidu/Projects/badminton-highlight-indexer/docs/HOW_RALLY_DETECTION_WORKS.md:23:WASB is
a CNN that, for each frame, outputs a heatmap ? the shuttle's `(x, y)` + a
C:/Users/avidu/Projects/badminton-highlight-indexer/docs/PMF_MARKET_RESEARCH.md:140:|
**BadPro** | **Badminton AI on phone** | **Pay-per-use, no sub** | Per-analysis (? "< one
coaching session") | Direct badminton competitor; shot detection, heatmaps, auto-highlights
(2025) |

```

21. assistant (2026-06-13T19:14:59.155Z)

This is the smoking gun that REFUTES the finding. The memory notes (current-state.md:1432-1436) and CODE_MAP.md:230 both explicitly document the WASB coordinate handling:

```

- current-state.md:1435: "run_tensor and used to **map heatmap coords back to original frame**"
- current-state.md:1436: "per sliding window of 3 consecutive frames ? affine-resize to 512x288
? ... ? detector.run_tensor"
- CODE_MAP.md:230: "***affine resize maps heatmap coords -> original-frame coords**"

```

So the `trans` (affine transform matrix) passed as `batch[2]` into `detector.run\_tensor(imgs, trans)` (wasb_infer.py:297) is exactly the inverse transform that WASB applies internally to map detected heatmap coordinates **back to original-frame pixel coordinates**. The WASB detector's `run\_tensor` un-projects the heatmap-space detection back to original video pixels using `trans`.

Let me confirm this by reading the `trans` handling in `wasb\_infer.py` and CODE_MAP more carefully to be airtight.

22. user (2026-06-13T19:15:00.112Z)

```

224    § **observability report** (`reporting/harvest.record_run` -> `RunRecorder`): stages
`validation/segmentation/ai_handoff` + highlights + verdict ? {pass,pass_with_warnings,fail};
flags `{code,faq_ref}`; `video_meta.fps_source` ? {probed,fallback}`. ? stage/metric/detail names
are coupled to `spec.yaml::rules[].when[].path` ? pairs that MUST stay in sync:
`ai_handoff.details.{api_key_present,ai_based}`, `segmentation.metrics.segment_count`, `segmenta

```

```

tion.details.{segmenter_actual,matches_config_default,segmenter_explicitly_requested,was_reinges
t}`, `validation.status`, `video_meta.fps_source`.
225
226 ---
227
228 ## 4. WASB WSL CONTRACT (external dep `~/models/WASB-SBDT`, MIT; NOT in this repo ?
claims below are unverified against external source, only the repo-side caller wasb_infer.py was
checked)
229 - Env: WSL conda `wasb`, cwd `~/models/WASB-SBDT/src`. Hydra
`compose(config_name='eval', overrides=[dataset=<sport>, model=wasb,
detector.model_path=<weights>, runner.device=cuda, runner.gpus=[0]])`. `gpus=[0]` because box is
single-GPU. (overrides VERIFIED in @backend/pipeline/detectors/wasb_infer.py::_build_cfg.)
230 - `model=wasb` -> `name=hrnet`, `frames_in=3`, `frames_out=3`, input 512x288 (WxH),
ImageNet normalize; affine resize maps heatmap coords -> original-frame coords. (model dims read
from cfg["model"] at runtime in wasb_infer.run.)
231 - `detectors/detector.py::TracknetV2Detector.run_tensor(imgs, affine_mats) -> (results,
hms_vis)`. `results[bid][eid]` = list of `{xy:np.array (ORIGINAL coords), score, scale}`.
(caller uses `detector.run_tensor(imgs, trans)` returning `(batch_results, _)` ? consistent.)
232 - `trackers/online.py::OnlineTracker` ? STATEFUL, sequential per `update()` (assumes
every frame fed once, in order, no gaps). `update(frame_dets)->{x,y,visi,score}`; `refresh()`
resets. `det['xy']` MUST be np.array. Full ordered re-run is deterministic. (caller calls
`build_tracker(cfg)`, `tracker.refresh()`, `tracker.update(...)` returning dict with `visi/x/y`
? consistent.)
233 - `dataloaders/dataset_loader.py::ImageDataset(cfg, samples, input_wh, output_wh,
transform, seq_transform, is_train)`; `samples=[{frames:[paths], annos:[{frame_path,
center:Center(is_visible,x,y)}]]`. (caller constructs exactly this ? VERIFIED in
wasb_infer.run.)
234 - Weights `pretrained_weights/wasb_badminton_best.pth.tar` ? NOT committed, academic-
data-trained -> ? confirm terms before COMMERCIAL deploy. monotrack weights = noncommercial,
avoided.
235
236 ---
237
238 ## 5. CROSS-REPO (sibling private repo `sports-data-collector` ? external, unverified
here)
239 - Indexer keys video by **FILE MD5**; collector keys by human `video_id`. ? re-
encoding/re-downloading changes bytes -> MD5; acquisition MUST precede processing; manifest
tracks the exact file used.
240 - `python -m src.cli.curate export` (collector) -> `<out>/<sport>/<video_id>.csv`
(indexer `start,end` format) + `manifest.json` -> consumed by `@backend/eval/batch.py` +
`@run_phase3_eval.py`.
241 - Collector also emits golden rally labels (`rally_number,start_time,end_time,...`) ->
consumed by `@backend/eval/calibrate_wasb.py` / `export_wasb_segments.py` /
`@backend/tools/rally_slicer.py`.

```

23. user (2026-06-13T19:15:01.027Z)

```

1428 AI handoff untested (wasb_infer core IS validated on real frames). config.json needs
an
1429 `indexing.wasb` section (defaults exist in WasbConfig). (4) per-video calibration
**Workflow**
1430 = DEFERRED (user will call it). Calibration only needs Phase-2 windows (no AI
handoff/API).
1431 Preprocessing (RESOLVED): `configs/model/wasb.yaml` ? name=hrnet, frames_in=3,
frames_out=3,
1432 inp 512x288 (WxH). `dataloaders/__init__.py build_img_transforms`: T.ToTensor +
Normalize
1433 mean[0.485,0.456,0.406]/std[0.229,0.224,0.225] (ImageNet). Resize via affine
(`utils.image.
1434 get_affine_transform` ? input_wh; the affine matrix is the `trans`/`affine_mats`
passed to
1435 run_tensor and used to map heatmap coords back to original frame). So wrapper: per
sliding window
1436 of 3 consecutive frames ? affine-resize to 512x288 ? ToTensor+Normalize ? stack ?
detector.run_tensor

```

1437 ? tracker ? (x,y,visi,score) per frame ? CSV Frame,Visibility,X,Y. Weight variant:
 wasb_badminton_best.

1438

1439 ## GOAL CORRECTION (user clarified 2026-06-06 ? important)

1440 User wants to "make the WASB pretrained WEIGHTS more robust with the 1 annotated video."
 Honest reality:

1441 - Our annotation = **temporal rally [start,end]** labels, NOT per-frame shuttle (x,y).
 WASB is a shuttle

1442 DETECTOR (needs x,y heatmap supervision). **So the rally labels CANNOT fine-tune
 WASB's detector weights.**

1443 - What the 1 video CAN quantifiably improve = the **rally-SEGMENTATION layer on top of
 WASB**: the 3

1444 windowing thresholds (velocity_thresh/merge_gap/min_window_duration) + WASB variant
 choice. Measure

1445 IoU/F1 on HELD-OUT golden rallies (calibrate on K, test on rest). That's the
 "quantifiable improvement".

1446 - To genuinely fine-tune WASB DETECTOR weights ? need per-frame shuttle (x,y) labels (a
 different

1447 annotation, or pseudo-labels) ? separate track, not this annotation.

24. assistant (2026-06-13T19:15:11.484Z)

This is definitive. CODE_MAP.md:231 documents the WASB-SBDT detector contract directly:

```
> `detectors/detector.py::TracknetV2Detector.run_tensor(imgs, affine_mats) -> (results,
hms_vis)`. `results[bid][eid]` = list of `{xy:np.array (**ORIGINAL coords**), score, scale}`.
```

And current-state.md:1433-1435 documents that the affine matrix (`trans`/`affine\_mats`) passed
 to `run\_tensor` "is used to **map heatmap coords back to original frame**."

So in `wasb\_infer.py`:

- Line 297: `batch\_results, \_ = detector.run\_tensor(imgs, trans)` ? the detector consumes the
 affine transform `trans` and returns `xy` already **in original-frame coordinates**, not heatmap
 space.
- Line 304: `\_det\_plain(d)` extracts `det["xy"]` ? original coords ? into the cache.
- Line 338-339: the tracker processes these original-coord detections and returns
 `r.get('x')`/`r.get('y')` ? still original coords.

The reviewer's core technical premise ? that WASB X/Y "operate in the model's OUTPUT heatmap
 resolution (`output\_wh`)" ? is **incorrect**. The `output\_wh` (line 223) is passed to
 `ImageDataset` only to construct the training/eval heatmap target shape; the inference-time
 detection is un-projected back to original pixels by the affine transform inside `run\_tensor`.
 The shuttle X/Y and person bboxes are therefore in the **same** original-video pixel space, and
 `\_shuttle\_in\_box`'s direct pixel comparison is correct.

Let me do one more sanity check to make sure `output\_wh` isn't used anywhere to scale the final
 coords on the repo side, and confirm `\_det\_plain` / `\_dets\_to\_tracker` don't rescale.

25. user (2026-06-13T19:15:15.631Z)

```
backend\pipeline\detectors\wasb_infer.py:223: output_wh = (int(cfg["model"]["out_width"]),
int(cfg["model"]["out_height"]))
backend\pipeline\detectors\wasb_infer.py:278: ds = ImageDataset(cfg, remaining, input_wh,
output_wh,
```

26. assistant (2026-06-13T19:15:22.250Z)

Confirmed. `output\_wh` is used in exactly one place on the repo side: passed to `ImageDataset`
 constructor (line 278) ? which uses it to build the heatmap *target* tensor shape for the
 dataset, NOT to scale the output coordinates. It never touches the `xy` coordinates that flow
 into the trajectory CSV. There is no rescaling because none is needed: the affine transform
 inside `run\_tensor` already maps detections to original-frame pixels.

The reviewer's own caveat acknowledged the uncertainty: "confirming the exact WASB output scale

requires the WASB-SBDT repo in WSL (not in this repo), so this is GPU/WSL-side to fully verify." The repo's own documented external contract (CODE_MAP.md §4, which is the authoritative cold-start map verified against `run\_tensor`'s caller) resolves that uncertainty against the finding: WASB returns ORIGINAL coordinates.

I have enough to render a verdict.

27. user (2026-06-13T19:15:42.800Z)

Structured output provided successfully

28. assistant (2026-06-13T19:15:53.271Z)

I verified the finding against the actual code and the repo's documented WASB contract. Verdict: ****not real (false positive)****.

The reviewer's core claim is that WASB shuttle X/Y are emitted in the model's output-heatmap resolution (~512x288) while person bboxes are in original-video pixels, so `shuttle\_player\_colocation` would always be ~0. That premise is wrong:

- `docs/CODE\_MAP.md:231` documents the external detector contract: `TracknetV2Detector.run\_tensor(imgs, affine\_mats)` returns `{xy: np.array (ORIGINAL coords), ...}` ? the affine transform un-projects heatmap detections back to original-frame pixels.
- `backend/pipeline/detectors/wasb\_infer.py:297` passes `trans` (the inverse affine) into `run\_tensor`, so `det["xy"]` (line 304) and the tracker's `r.get("x")/r.get("y")` (lines 338-339) are already in original pixels. Nothing in the repo rescales to heatmap space.
- `output\_wh` (line 223) is used in exactly one place, line 278, to size the heatmap *target* tensor inside `ImageDataset` ? it never touches the output coordinates.

Corroboration: the shipped, calibrated WASB substrate (`tracknet\_runner.trajectory\_to\_action\_windows`) normalizes shuttle velocity by `frame\_width`, and `calibrate\_wasb.py`/`export\_wasb\_segments.py` default `--frame-width 1920` (original width). If the X/Y were 512-space, the production WASB rally detector itself would be on a mismatched scale.

The reviewer self-flagged the key uncertainty ("requires the WASB-SBDT repo in WSL... GPU/WSL-side to fully verify"). The repo's authoritative cold-start map resolves it against the finding. The conceded part (temporal frame-index alignment is correct) holds; the load-bearing spatial-mismatch claim does not.

Residual: the external WASB-SBDT source isn't in this repo, so ground truth is one documented layer removed ? a one-line shared-space comment in `\_shuttle\_in\_box` and an owner real-clip spot check would close that low residual, but no code fix is warranted.